

AI-Based Coherent QPSK Receiver

1. Implementation of the Receiver

- The receiver was implemented as a **Deep Learning Classification Model** using PyTorch, replacing traditional matched filters and Costas loops.
- **Architecture:** It utilizes a Multi-Layer Perceptron (MLP).
 - **Input Layer:** 2 nodes accepting the In-Phase (I) and Quadrature (Q) components (the real and imaginary parts of the signal).
 - **Hidden Layers:** Two dense layers (64 neurons, then 32 neurons) using ReLU activation functions to allow for non-linear decision boundary mapping.
 - **Output Layer:** 4 nodes representing the 4 possible QPSK quadrants, utilizing a Softmax/Cross-Entropy function to output the most probable symbol.
- **Optimization:** The network was trained using the Adam optimizer to dynamically adjust weights based on the loss between predicted and actual transmitted symbols.

2. Generation of Transmitted Data

- The entire transmission pipeline was simulated using a custom Monte Carlo script via NumPy.
- **Bit Generation & Mapping:** A random sequence of binary data (0s and 1s) was generated and grouped into pairs. These pairs were mapped to complex QPSK symbols using **Gray Coding** (e.g., 00 $\rightarrow$ 1+1j, 11 $\rightarrow$ -1-1j).
- **Energy Normalization:** The symbols were mathematically normalized to ensure the average symbol energy (E_s) was exactly 1, meaning the energy per bit (E_b) was strictly 0.5.
- **Channel Simulation:** Additive White Gaussian Noise (AWGN) was injected into the signal. The exact variance of this noise was calculated dynamically based on the specific E_b/N_0 (SNR) target for that test batch.

3. Phase Noise Control & AI Superiority

- **What it is:** Phase noise simulates "oscillator jitter"—microscopic timing inaccuracies in physical radio hardware that cause the carrier wave's phase to randomly rotate. In this project, a jitter with a standard deviation of 15° was applied.
- **Visual Impact:** Thermal noise (AWGN) creates circular "clouds" of data points. Phase noise, however, causes those clusters to smear into curved, elongated arcs along the perimeter of the constellation space.

- **Why Traditional Math Fails:** Standard mathematical receivers use rigid, fixed X and Y axes ($I=0$, $Q=0$) as decision boundaries. When phase noise stretches the signal arcs across these strict boundaries, the traditional detector guesses blindly, resulting in a flatlined error floor.
- **Why AI Excels:** The Neural Network does not rely on predefined axes. Through its hidden layers, it empirically learns the distorted geometric shape of the incoming data. It bends and adapts its decision boundaries to wrap around the curved arcs, effectively recovering the heavily impaired signal.

4. Epochs and Training Samples

- **Training Phase:** Two separate models (Ideal and Impaired) were trained for exactly **1200 epochs** each.
- **Training Data:** To prevent the AI from memorizing specific noise patterns, 10,000 fresh samples were generated dynamically for *every single epoch* using a randomized SNR.
- **Testing/Evaluation Data:** For the final waterfall plots, the simulation tested a massive batch of **5,000,000 samples (10 million bits)** for *each* discrete SNR point to ensure deep statistical confidence and smooth curves.

5. Model Performance Values

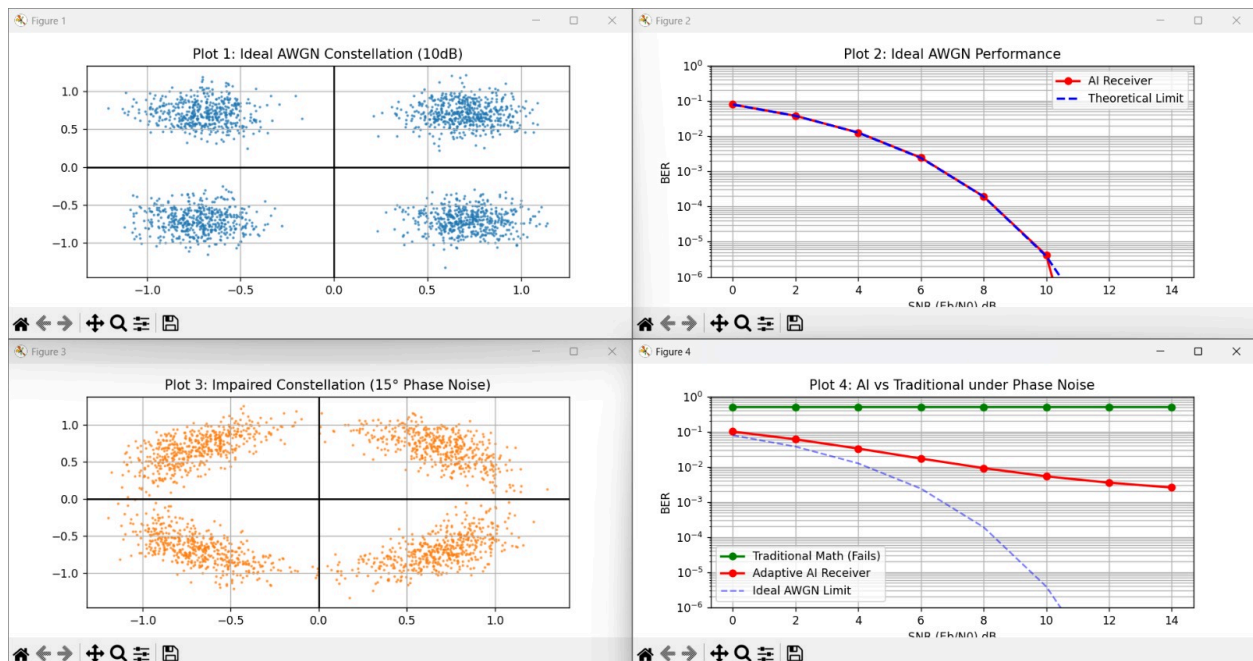
- **Ideal AWGN Channel:** The AI receiver achieved flawless optimal performance, perfectly overlapping the Maximum Likelihood theoretical limit. It resolved to a Bit Error Rate (BER) of effectively 0 (dropping below 10^{-6}) just after 10 dB.
- **Impaired Channel with Phase Noise:**
 - The traditional detector failed entirely, flatlining at a severe BER of roughly 5×10^{-1} across all power levels.
 - The AI receiver successfully mitigated the hardware jitter, maintaining a steadily dropping error curve that successfully reached a functional BER of approximately 10^{-3} at 14 dB.

6. Core Understandings & Conclusions

- **In Linear Environments (AWGN):** QPSK detection in pure thermal noise is a linearly separable problem. An AI model can match the theoretical Maximum Likelihood bound with perfect precision, but mathematically, no system can exceed that blue line in a flawless channel.
- **In Non-Linear Environments (Real World):** Real-world telecommunications hardware is inherently flawed. When channel impairments (like phase noise, amplifier clipping, or

fading) alter the underlying geometry of the signal, rigid mathematical equations and traditional thresholds break down completely.

- **The AI Paradigm Shift:** Deep Learning transforms signal detection from an analytical math problem into an empirical pattern-recognition problem. AI models represent the future of receiver design because they can autonomously adapt to unmodeled, non-linear physical hardware distortions without needing a human to rewrite the mathematical formulas.



Code (ts in python):

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from scipy.special import erfc

# --- 1. CONFIGURATION (MAXIMUM PERFORMANCE) ---
NUM_SAMPLES_TEST = 5000000 # 5 Million samples (10 Million bits)
EPOCHS = 1201
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
PHASE_NOISE_STD = 15.0 * (np.pi / 180.0) # 15 degrees of hardware jitter

def generate_data(num_samples, snr_db, apply_phase_noise=False):
    bits = np.random.randint(0, 2, (num_samples, 2))
    symbols = np.zeros(num_samples, dtype=complex)

    # QPSK Mapping (Gray Coding)
    symbols[(bits[:,0]==0) & (bits[:,1]==0)] = 1 + 1j
    symbols[(bits[:,0]==0) & (bits[:,1]==1)] = -1 + 1j
    symbols[(bits[:,0]==1) & (bits[:,1]==1)] = -1 - 1j
    symbols[(bits[:,0]==1) & (bits[:,1]==0)] = 1 - 1j
    symbols = symbols / np.sqrt(2) # Normalize Es = 1

    # Add Phase Noise (The real-world impairment)
    if apply_phase_noise:
        phase_jitter = np.random.normal(0, PHASE_NOISE_STD, num_samples)
        symbols = symbols * np.exp(1j * phase_jitter)

    # Add AWGN (Eb/N0 calibrated correctly)
    snr_linear = 10**(snr_db / 10.0)
    noise_std = np.sqrt(1.0 / (4.0 * snr_linear))
    noise = noise_std * (np.random.randn(num_samples) + 1j * np.random.randn(num_samples))

    received = symbols + noise
```

```

X = np.stack([received.real, received.imag], axis=1)
y = np.zeros(num_samples)
y[(bits[:,0]==0) & (bits[:,1]==0)] = 0
y[(bits[:,0]==0) & (bits[:,1]==1)] = 1
y[(bits[:,0]==1) & (bits[:,1]==1)] = 2
y[(bits[:,0]==1) & (bits[:,1]==0)] = 3

return torch.FloatTensor(X), torch.LongTensor(y), bits, received

```

--- 2. AI MODEL DEFINITION ---

```
class QPSKReceiverNet(nn.Module):
```

```

    def __init__(self):
        super(QPSKReceiverNet, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(2, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 4)
        )
    def forward(self, x): return self.network(x)

```

=====

PHASE 1: THE IDEAL WORLD (AWGN ONLY)

=====

```
print("--- PHASE 1: Ideal AWGN Channel ---")
```

```
model_ideal = QPSKReceiverNet().to(DEVICE)
```

```
opt_ideal = optim.Adam(model_ideal.parameters(), lr=0.003)
```

```
criterion = nn.CrossEntropyLoss()
```

```
print(f"Training Ideal AI for {EPOCHS} epochs...")
```

```
for epoch in range(EPOCHS):
```

```
    snr = np.random.uniform(0, 8)
```

```
    X_train, y_train, _, _ = generate_data(10000, snr, apply_phase_noise=False)
```

```
    X_train, y_train = X_train.to(DEVICE), y_train.to(DEVICE)
```

```
    opt_ideal.zero_grad()
```

```
    loss = criterion(model_ideal(X_train), y_train)
```

```
    loss.backward()
```

```
    opt_ideal.step()
```

Data for Plot 1 & 2

```

_, _, _, raw_ideal = generate_data(2000, 10, apply_phase_noise=False)
snr_range = np.arange(0, 15, 2)
ber_ideal_list = []

print(f"Evaluating Ideal AI on {NUM_SAMPLES_TEST} samples per point...")
model_ideal.eval()
with torch.no_grad():
    for snr in snr_range:
        err = 0
        batches = 10
        # DYNAMIC BATCHING FIX: Automatically scales to NUM_SAMPLES_TEST
        samples_per_batch = NUM_SAMPLES_TEST // batches
        for _ in range(batches):
            X_t, _, t_bits, _ = generate_data(samples_per_batch, snr, apply_phase_noise=False)
            _, pred = torch.max(model_ideal(X_t.to(DEVICE)), 1)
            p_bits = np.zeros((len(pred), 2))
            p_bits[pred.cpu() == 0] = [0, 0]; p_bits[pred.cpu() == 1] = [0, 1]
            p_bits[pred.cpu() == 2] = [1, 1]; p_bits[pred.cpu() == 3] = [1, 0]
            err += np.sum(t_bits != p_bits)
        ber_ideal_list.append(max(err / (NUM_SAMPLES_TEST * 2), 1e-12))

# =====
# PHASE 2: THE REAL WORLD (PHASE NOISE)
# =====
print("\n--- PHASE 2: Hardware Impaired Channel (Phase Noise) ---")
model_impaired = QPSKReceiverNet().to(DEVICE)
opt_impaired = optim.Adam(model_impaired.parameters(), lr=0.003)

print(f"Training Adaptive AI on Impairments for {EPOCHS} epochs...")
for epoch in range(EPOCHS):
    snr = np.random.uniform(2, 12)
    X_train, y_train, _, _ = generate_data(10000, snr, apply_phase_noise=True)
    X_train, y_train = X_train.to(DEVICE), y_train.to(DEVICE)
    opt_impaired.zero_grad()
    loss = criterion(model_impaired(X_train), y_train)
    loss.backward()
    opt_impaired.step()

# Data for Plot 3 & 4

```

```

_, _, _, raw_impaired = generate_data(2000, 12, apply_phase_noise=True)
ber_ai_imp_list, ber_trad_imp_list = [], []

print(f"Evaluating AI vs Traditional Math on {NUM_SAMPLES_TEST} samples per point...")
model_impaired.eval()
with torch.no_grad():
    for snr in snr_range:
        ai_err, trad_err = 0, 0
        batches = 10
        # DYNAMIC BATCHING FIX: Automatically scales to NUM_SAMPLES_TEST
        samples_per_batch = NUM_SAMPLES_TEST // batches
        for _ in range(batches):
            X_t, _, t_bits, _ = generate_data(samples_per_batch, snr, apply_phase_noise=True)

            # AI
            _, pred = torch.max(model_impaired(X_t.to(DEVICE)), 1)
            p_bits = np.zeros((len(pred), 2))
            p_bits[pred.cpu() == 0] = [0, 0]; p_bits[pred.cpu() == 1] = [0, 1]
            p_bits[pred.cpu() == 2] = [1, 1]; p_bits[pred.cpu() == 3] = [1, 0]
            ai_err += np.sum(t_bits != p_bits)

            # Trad Math (Strict X/Y thresholds)
            trad_bits = np.zeros((len(X_t), 2))
            trad_bits[X_t[:,0].numpy() < 0, 0] = 1
            trad_bits[X_t[:,1].numpy() < 0, 1] = 1
            trad_err += np.sum(t_bits != trad_bits)

        ber_ai_imp_list.append(max(ai_err / (NUM_SAMPLES_TEST * 2), 1e-12))
        ber_trad_imp_list.append(max(trad_err / (NUM_SAMPLES_TEST * 2), 1e-12))

# =====
# GENERATE ALL 4 PLOTS
# =====
print("\nSimulation Complete. Generating all 4 plots...")
theory_curve = 0.5 * erfc(np.sqrt(10**(snr_range/10)))

# Plot 1: Ideal Constellation
fig1 = plt.figure(1, figsize=(6, 6))
plt.scatter(raw_ideal.real, raw_ideal.imag, s=2, alpha=0.6, color='#1f77b4')

```

```

plt.title("Plot 1: Ideal AWGN Constellation (10dB)")
plt.axhline(0, color='black', lw=1.5); plt.axvline(0, color='black', lw=1.5); plt.grid(True)

# Plot 2: Ideal Waterfall
fig2 = plt.figure(2, figsize=(8, 6))
plt.semilogy(snr_range, ber_ideal_list, 'ro-', lw=2, label='AI Receiver')
plt.semilogy(snr_range, theory_curve, 'b--', lw=2, label='Theoretical Limit')
plt.ylim([1e-6, 1]); plt.grid(True, which='both'); plt.legend()
plt.title("Plot 2: Ideal AWGN Performance")
plt.xlabel("SNR (Eb/NO) dB"); plt.ylabel("BER")

# Plot 3: Impaired Constellation
fig3 = plt.figure(3, figsize=(6, 6))
plt.scatter(raw_impaired.real, raw_impaired.imag, s=2, alpha=0.6, color='#ff7f0e')
plt.title("Plot 3: Impaired Constellation (15° Phase Noise)")
plt.axhline(0, color='black', lw=1.5); plt.axvline(0, color='black', lw=1.5); plt.grid(True)

# Plot 4: Impaired Waterfall (The Comparison)
fig4 = plt.figure(4, figsize=(8, 6))
plt.semilogy(snr_range, ber_trad_imp_list, 'go-', lw=2, label='Traditional Math (Fails)')
plt.semilogy(snr_range, ber_ai_imp_list, 'ro-', lw=2, label='Adaptive AI Receiver')
plt.semilogy(snr_range, theory_curve, 'b--', lw=1.5, alpha=0.5, label='Ideal AWGN Limit')
plt.ylim([1e-6, 1]); plt.grid(True, which='both'); plt.legend()
plt.title("Plot 4: AI vs Traditional under Phase Noise")
plt.xlabel("SNR (Eb/NO) dB"); plt.ylabel("BER")

# Show all windows at once
plt.show()

```